# Part 2 Usage Guide: Design Models & Equilibrium

This directory provides the tools necessary to linearize a nonlinear system and verify its equilibrium. This is the foundation for all control design in Parts 3 and 4.

## Table of Contents


1. Workflow for Part 2
2. Finding Equilibrium ($u_e$)
3. Numerical Linearization ($A, B, C, D$)
4. State-Space to Transfer Function ($P(s)$)
5. Verification & Obfuscated Fallback


---

## 1. Workflow for Part 2

1. **Verify Dynamics**: Ensure your Part 1 implementation is correct by running the verification tool.
2. **Define** $\theta_e$: Identify the equilibrium angle requested (e.g., $0°, 30°, 90°$).
3. **Find** $u_e$: Calculate the torque required to hold the system at $\theta_e$.
4. **Linearize**: Generate the $A, B, C, D$ matrices around that point.
5. **Convert to TF**: Generate the transfer function for PID tuning.

## 2. Finding Equilibrium ($u_e$)

In `design_model_tool.py`, the `find_equilibrium` method solves:

$$f(x_e, u_e) = 0$$

For the Rod-Mass system, this means finding the torque $\tau_e$ that perfectly cancels out the gravity and spring forces at a specific angle $\theta_e$. * **Action**: Set `theta_e` in the script and run. The output `u_e` is your answer for Question 2.1.

## 3. Numerical Linearization ($A, B, C, D$)

Instead of doing messy partial derivatives by hand, the `compute_ss_matrices` method uses the **Finite Difference** method:

$$A \approx \frac{f(x_e + \epsilon, u_e) - f(x_e - \epsilon, u_e)}{2\epsilon}$$

* **Action**: The script automatically generates these matrices. * **Manual Check**: For a standard pendulum/rod: * $A[0, 1]$ should be 1. * $A[1, 0]$ represents the "stiffness" (spring + gravity effects). * $A[1, 1]$ represents the "damping" (friction $b$).

## 4. State-Space to Transfer Function ($P(s)$)

The `get_transfer_function` method uses the `control` library to compute:

$$P(s) = C(sI - A)^{-1}B + D$$

* **Action**: Copy the resulting polynomial from the script output for Question 2.4.

## 5. Verification & Obfuscated Fallback

**Testing your implementation:**

Run `python design_model_tool.py`. It will print: `SUCCESS: Your dynamics implementation matches the verified version!` If it fails, check your mass, length, or trig functions (sin vs cos) in `dynamics.py`.

**Using the Compiled Fallback:**

If you cannot fix your Part 1 implementation during the final, swap the import in any simulation file:

```
# From:
from L_rodmass.dynamics import RodMassDynamics as Dynamics
# To:
from L_rodmass.dynamics_compiled import RodMassDynamics as Dynamics
```

This allows you to complete Parts 2, 3, and 4 even if your Part 1 physics are broken.